Applying Attribution-based Parameter Decomposition to the L^4-trained CiS toy model

Francisco Ferreira da Silva
(autonomous run, Claude Opus 4.7)

2026-04-17

Abstract

We apply Attribution-based Parameter Decomposition (APD) [1] to our L^4-trained CiS toy models, both plain ($y = W_{\text{out}}\,\text{ReLU}(W_{\text{in}}x)$) and with a random Gaussian embedding. The APD paper shows APD recovering per-feature "mechanisms" in a similar compressed-computation architecture where each feature's target is distinct ($y_i = x_i + \text{ReLU}(c_i\,x_i)$ with random c_i). On our L^4 models, where features have identical targets ($y_i = \text{ReLU}(x_i)$) and the learned mechanism is combinatorial ReLU coding, APD does *not* recover per-feature components: the input and output halves of each APD component decouple onto disjoint index sets, so no single parameter component implements a feature's read-and-write mechanism. As a sanity check we retrain a paper-style target (with random per-feature coefficients) using the same architecture and find APD does produce partial per-feature alignment. We attribute the contrast to our network's mechanism being *per-codeword* rather than per-feature: multiple features route through the same neurons, so there is no per-feature circuit to recover.

1 Setup

1.1 APD in brief

APD [1] trains a second, "decomposed" copy of a target model whose weight matrices W are replaced by a sum of C rank-m components:

$$W = \sum_{c=1}^{C} A_c B_c, \qquad A_c \in \mathbb{R}^{d_{\text{in}} \times m}, \quad B_c \in \mathbb{R}^{m \times d_{\text{out}}}.$$

Training uses four losses:

- **param-match:** $\| \sum_c A_c B_c - W_{\text{target}} \|^2$ per layer, so the full sum reconstructs the target's weight.
- **topk-recon:** at each step, a gradient-attribution score is computed per component and a top-k subset is kept; the decomposed model is run through only those components and its output is pushed to match the target's output.
- **act-recon:** same as above but on the post-MLP-in activations (intermediate representations).
- **Schatten regulariser:** encourages components to be low rank.

Together these push the decomposition to be sparse (few components active per input), reconstructive (the sum equals the original weight), and functional (the top-k components can reproduce the target output).

1.2 Target models

We decompose five target models (all using our CiS architecture $y = W_{\text{out}}\,\text{ReLU}(W_{\text{in}}x)$ without residual connection and trained under L^4 loss):

- `plain_20f_5n`: $F = 20$ features, $n = 5$ neurons.
- `plain_20f_2n`: $F = 20$, $n = 2$.
- `plain_100f_10n`: $F = 100$, $n = 10$.
- `embed_20f_5n_D80`: embedded variant with $D = 80$, otherwise like `plain_20f_5n`.
- `embed_20f_2n_D40`: embedded variant with $D = 40$, otherwise like `plain_20f_2n`.

As a *pipeline sanity check*, we also train a "paper-style" target that mimics the APD paper's toy-model task: target $y_i = \text{ReLU}(c_i\,x_i)$ with random $c_i \in [1, 2]$, same architecture and size as `plain_20f_5n`.

1.3 Adapting the APD code to our architecture

The APD paper uses a target architecture with a residual connection, $y = W_U(\text{resid} + \text{MLP}(\text{resid}))$. Our target has no residual. We apply a minimal monkey-patch to `ResidualMLPModel` and `ResidualMLPSPDModel` that replaces the residual add (`resid` $\leftarrow$ `resid + MLP(resid)`) with a pure substitution (`resid` $\leftarrow$ `MLP(resid)`). With this change we load our trained weights as the target and set W_E, W_U to identity for `plain_*` and to the fixed random E, E^T for `embed_*`. We verified numerically that the patched APD model reproduces our model's output to float precision (max difference $< 10^{-7}$ on 512 random sparse samples).

1.4 APD training configuration

Following the paper, we used:

- $C = 30$ components for $F = 20$ targets (20 features < 30 components, small overhead for "dying" components), $C = 130$ for $F = 100$.
- 10,000 steps, batch size 256, Adam lr 10^{-3}, cosine schedule.
- Gradient-based attribution, `batch_topk=True`, `topk=1.28` (paper value; slightly above expected active features per sample).
- `unit_norm_matrices=True`. Coefficients: param-match 1, topk-recon 1, act-recon 1, Schatten 10 with $p = 0.9$.
- Input distribution during APD training: $p = 0.05$ so that $E[\text{active per sample}] = 1$ (matching the paper's ratio of topk$/E[\text{active}] \approx 1.28$). For $F = 100$ we use $p = 0.02$, topk $= 2.56$.

2 Results

2.1 Pipeline sanity: paper-style target

On the paper-style target (per-feature random coefficients), APD recovers partial per-feature decomposition (Figure 1):

- 19/30 components on the input side are "dead" (essentially zero weight). Of the 11 alive, 7 are monosemantic (read strongly from a single feature).

- On the output side, 9/30 dead, 9 monosemantic, 10 duo.
- **2/30 components are alive on both sides with the same dominant feature** (the "joint monosemantic" criterion). These are clean per-feature mechanisms.
- Scrubbing experiment: ablating half the components while keeping those whose dominant feature is active in the current sample gives mean MSE 7.7×10^{-4}; doing the opposite (anti-scrubbing) gives 6.2×10^{-3}. Ratio $\approx 8\times$. The decomposition is functionally meaningful.

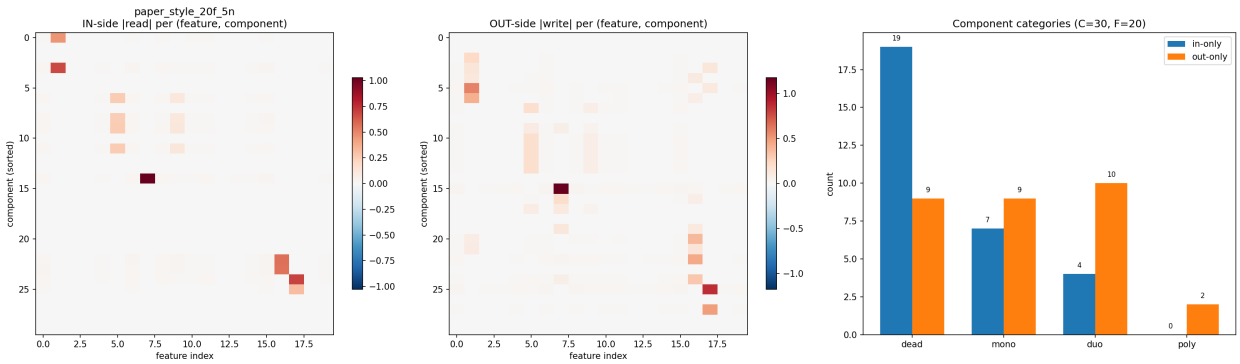

Figure 1: **APD on paper-style target.** Left: per-feature read strength of each input-side component (rows are components sorted by dominant feature). Middle: per-feature write strength of each output-side component. Right: component category counts, in-side vs out-side. Some components have a clear dominant feature on both sides, matching the paper's qualitative finding.

This is worse than the paper's reported result ($\sim 100/100$ features covered) but consistent with our smaller, more compressed task ($F/n = 4$ here vs 2 in the paper). It confirms that our APD pipeline is working and can in principle find per-feature components.

2.2 Main finding: APD on L^4 CiS models gives polysemantic components with decoupled in/out halves

On our L^4-trained plain and embedded models the picture is very different. Figures 2–3 show heatmaps from `plain_20f_5n` and `plain_20f_2n`:

- Many components are alive on the input side and many on the output side, but the alive sets are *disjoint*: almost no component has both halves alive simultaneously. For `plain_20f_5n` we get 15 alive on IN, 15 alive on OUT, and 0/30 alive on both.
- Each component that is alive on its one side is polysemantic – reads or writes multiple features. No component cleanly maps to a single feature's mechanism.
- The scrubbing/anti-scrubbing ratio is < 1: the "dominant feature" of a component is essentially random, so keeping "relevant" components doesn't preserve behaviour.

Table 1 summarises across all runs.

2.3 Why? Codeword sharing means no per-feature circuit

Our L^4-trained CiS model implements *combinatorial ReLU coding* (see companion report): features are routed to neurons via sign patterns of W_{in} columns ("codewords"), and many features can share a single codeword. When features i and j share a codeword, they pass through the exact same

Table 1: APD decomposition summary. "alive both" = components alive on both input and output sides. "aligned" = of those, how many have the same dominant feature on both sides. "anti/scrub" is the ratio of anti-scrubbed MSE to scrubbed MSE; > 1 means the decomposition is behaviourally meaningful.

target	F	C	alive_both	aligned	joint mono	anti/scrub	MSE full
paper_style_20f_5n	20	30	2	2	2	**8.05**	3.5×10^{-4}
plain_20f_5n	20	30	0	0	1	0.39	2.3×10^{-4}
plain_20f_2n	20	30	0	0	1	0.19	2.1×10^{-4}
plain_100f_10n	100	130	9	5	2	0.39	1.0×10^{-5}
embed_20f_5n_D80	20	30	5	2	2	0.75	7.0×10^{-5}
embed_20f_2n_D40	20	30	0	0	3	0.21	7.0×10^{-5}

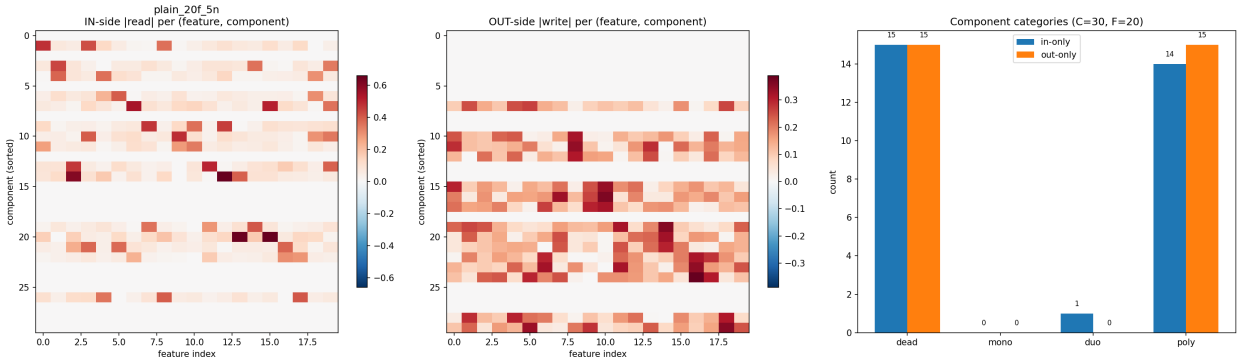

Figure 2: **APD on `plain_20f_5n`.** Both IN and OUT sides have many alive components, each covering multiple features (dense horizontal bands). The alive components on each side are at different indices: no component has a coherent "read feature j then write feature j" structure. Right panel: no monosemantic or duo components.

neurons, so the "mechanism" of feature i is identical to the mechanism of feature j. There is no per-feature circuit to recover.

APD's ansatz assumes features have dissociable mechanisms: each component should implement one feature's input-to-output map. In the paper-style target, random per-feature coefficients c_i create genuinely distinct per-feature mechanisms, so APD recovers them (partially). In our *uniform-target* setting ($y_i = \mathrm{ReLU}(x_i)$ for all i), features share circuits, and APD has no way to separate them – so it returns a polysemantic decomposition whose input and output halves are additionally decoupled.

The 20f/2n case makes this especially vivid: with only 3 codewords to cover 20 features, the OUT-side component heatmap shows each alive component writing to essentially all 20 features (wide horizontal bands). This is the signature of shared circuitry on the write side.

2.4 Plain vs embedded

The embedded variants (`embed_*`) show the same qualitative picture as the plain ones (Table 1). One small quantitative difference: `embed_20f_5n_D80` does have 5 components alive on both sides (vs 0 for the plain variant at the same F,n), and 2 of those are in/out aligned. Scrubbing ratio improves from 0.39 to 0.75 (still < 1, still not a meaningful decomposition, but visibly less pathological).

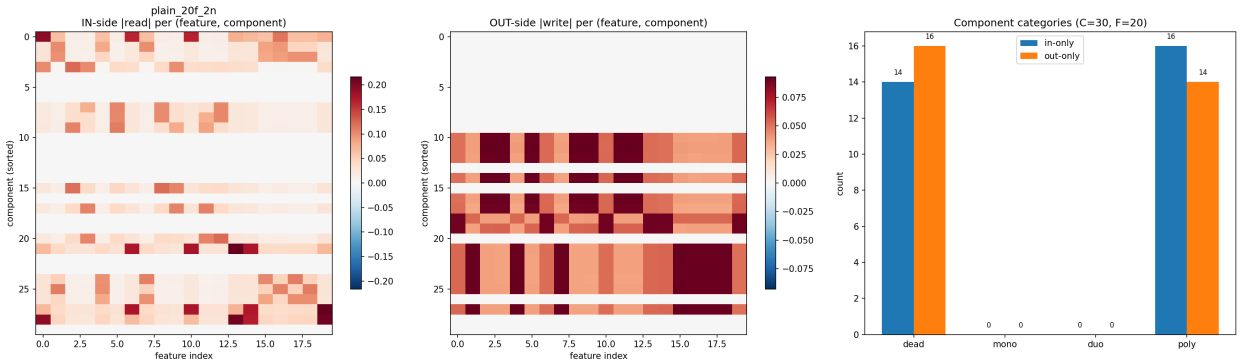

Figure 3: **APD on `plain_20f_2n`.** More extreme version of the same pattern. With only 3 codewords for 20 features, the OUT-side alive components each write to (nearly) all 20 features simultaneously – a direct signature of codeword sharing.

Our reading: the random embedding breaks some residual symmetry between input and output sides, which makes pairing slightly easier, but doesn't change the underlying story.

2.5 Scaling: `plain_100f_10n`

Our largest target has 100 features, 10 neurons. APD with $C = 130$ produces 121 joint-dead components and ~ 9 alive on both sides (5 aligned). Two features get a clean monosemantic component. The overall picture matches the $F = 20$ runs: polysemantic, decoupled in/out halves, no per-feature decomposition. Reconstruction MSE is 10^{-5} — better than at $F = 20$ because the model has more neurons per feature, so the decomposition sum is easier.

2.6 Anti/scrub ratio: a summary diagnostic

Table 1 compresses a lot of information into one column: the scrubbing ratio "anti/scrub". When this is > 1, APD has found a meaningful association between components and features: ablating the components attributed to active features (anti-scrub) does more damage than ablating unrelated ones (scrub). The paper-style target sits at 8.05. Our five L^4 targets all sit between 0.19 and 0.75, i.e. the "dominant feature" criterion over components is no better than random. This is the single cleanest summary of the null result.

3 Sanity-checks and caveats

- **Reconstruction works.** For every run the full-decomposition MSE (sum of all C components) is $\sim 10^{-4}$, essentially matching the target model's output. The decomposition isn't "broken" at the level of reconstructing weights; it just doesn't factorise into per-feature pieces.
- **Hyperparameter sensitivity.** We tried three `topk` regimes (`topk=1.0` at $p = 0.02$, topk $= 0.5$ at $p = 0.02$, and the final `topk=1.28` at $p = 0.05$) plus `data_generation_type="exactly_one_active"` and `exact_topk=True`. Qualitatively all variants gave polysemantic, decoupled-in-out decompositions for our L^4 targets. The paper-style target gave partially meaningful decomposition in all tried configurations.
- **Paper replicability.** Our 2/30 joint-monosemantic count on the paper-style target is lower than the paper's reported result. The difference likely comes from (a) tighter compression

($F/n = 4$ vs 2 in the paper), (b) no residual connection in our target (theirs can route some information around the MLP), and (c) a shorter, less-tuned run. We did not attempt to tune APD hyperparameters beyond matching the paper's config to our setup.

- **Scrubbing caveat.** Our scrubbing criterion uses the "dominant feature" of each component (argmax over per-feature attribution). When components are polysemantic this criterion is noisy; the < 1 ratios should be read as "no meaningful dominant feature" rather than "APD produces harmful ablations."

4 Conclusion

APD applied to our L^4-trained CiS toy models does *not* produce a clean per-feature decomposition. Components' input and output halves end up on disjoint sets of indices, and each alive component is polysemantic. A pipeline sanity check on a paper-style target confirms APD can produce at least partially per-feature components when the target has distinguishable per-feature mechanisms.

We interpret this as consistent with the mechanism we identify in the companion work: the L^4 model implements combinatorial ReLU coding, where multiple features share the same neurons via shared codewords. There is no per-feature circuit, so there is nothing per-feature for APD to recover. The L^2-trained ("naive") variants of the same architecture would presumably decompose cleanly per-feature, since they implement one feature per neuron; we didn't verify this here but it is a natural next experiment.

References

[1] Bushnaq *et al.* (2025), *Interpretability in Parameter Space: Minimizing Mechanistic Description Length with Attribution-based Parameter Decomposition*, arXiv:2501.14926.